

rfaR: Reservoir Frequency Analysis in R

User and Technical Manual

Dan McGraw, P.E. Andrew Verdin, PhD, P.H.
C. Haden Smith, P.E.

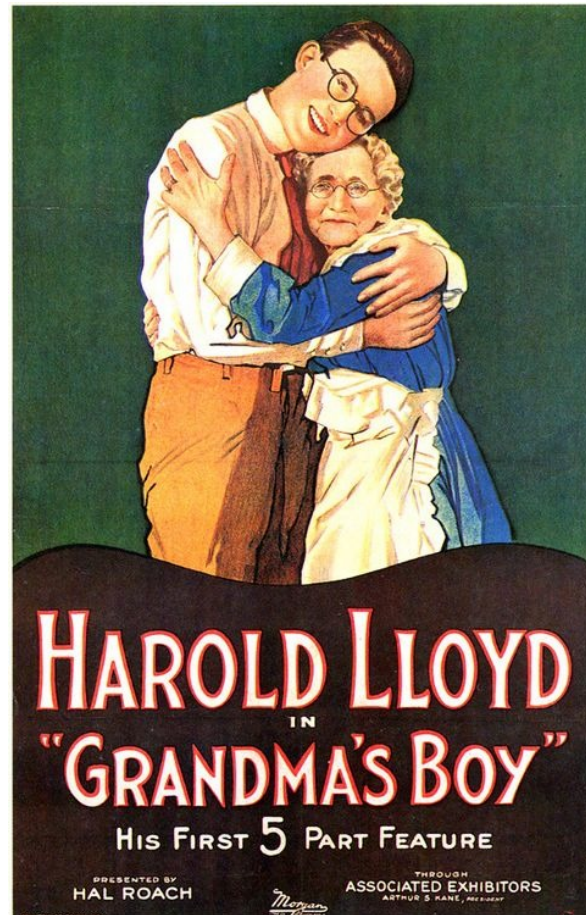
2026-08-14

Table of contents

Preface	4
How to cite	5
1 Introduction	6
1.1 Purpose	6
1.2 rfaR Analysis Summary	6
1.3 Relationship to RMC-RFA	6
1.4 How This Manual Is Organized	7
2 Methodology	8
2.1 Overview	8
2.2 Outer Loop: Parameter Uncertainty	8
2.3 Inner Loop: Realizations	8
2.4 Flood Routing	8
3 Installation	9
3.1 Requirements	9
3.2 Installing rfaR	9
3.3 Dependencies	9
3.4 Verifying Installation	9
4 Workflow	10
4.1 Overview	10
4.2 Step 1: Prepare Inputs	10
4.3 Step 2: Hydrograph Setup	10
4.4 Step 3: Sampling	10
4.5 Step 4: Routing	10
4.6 Step 5: Running the Full Simulation	10
4.7 Step 6: Interpreting Results	10
5 Function Reference	11
5.1 Execute Reservoir Frequency Analysis	11
5.1.1 <code>rfa_simulate()</code>	11
5.2 Primary RFA Modules	17
5.2.1 <code>stratified_sampler()</code>	17
5.2.2 <code>flow_frequency_sampler()</code>	19

5.2.3	<code>flow_frequency_sampler_expected()</code>	20
5.2.4	<code>hydrograph_setup()</code>	22
5.2.5	<code>scale_hydrograph()</code>	24
5.2.6	<code>mod_puls_routing()</code>	26
5.2.7	<code>stage_frequency_curve()</code>	27
5.3	Rejection Sampling	28
5.3.1	<code>pmf_stage_lognormal()</code>	28
5.3.2	<code>rejection_sampling_stage()</code>	30
5.4	Additional Utilities	31
5.4.1	<code>bootstrap_vfc()</code>	31
5.4.2	<code>qp3()</code>	33
5.4.3	<code>aep2stage()</code>	34
5.4.4	<code>stage2aep()</code>	35
5.4.5	<code>interpolate_aep_matrix()</code>	36
5.4.6	<code>interpolate_stage_matrix()</code>	37
5.4.7	<code>power_function()</code>	37
5.4.8	<code>theme_rfar_conceptual()</code>	38
6	Simulation Modes	40
6.1	Median Mode	40
6.2	Expected Mode	40
6.3	Full Uncertainty Mode	40
6.4	Selecting a Mode	40
7	Validation	41
7.1	Purpose	41
7.2	Modified Puls Routing	41
7.3	Stratified Sampling	41
7.4	See Also	41

Preface



`rfaR` implements the RMC Reservoir Frequency Analysis (RMC-RFA) methodology in R, producing reservoir stage-frequency curves with uncertainty bounds by combining deterministic Modified Puls flood routing with a nested Monte Carlo framework.

This manual complements the [function reference site](#) generated by pkgdown. Where the pkgdown site documents individual functions, this manual explains the methodology, workflow, and how the pieces fit together.

How to cite

Cover image: “Grandma’s Boy” (1922) movie poster, public domain, via [Wikimedia Commons](#).

1 Introduction

1.1 Purpose

`rfaR` is an R implementation of the Risk Management Center Reservoir Frequency Analysis (RMC-RFA) software. RMC-RFA (and `rfaR`) use apply stratified monte carlo methods to deterministic flood routing to produce a hydrologic hazard curve¹ at individual dam and levee sites. Hazard analyses support risk assessments within the USACE Dam & Levee Safety Program.

1.2 rfaR Analysis Summary

`rfaR` combines deterministic flood routing (Modified Puls) with a nested, two-loop Monte Carlo framework to quantify:

- **Natural variability** — simulated in the inner loop as a *realization*. Each realization represents 10,000 individual flood events with randomly sampled starting stage, hydrograph shape, and inflow volume (from an associated frequency distribution).
- **Knowledge uncertainty (epistemic uncertainty)** — simulated in the outer loop, by sampling different parameter sets (from LP3 or GEV flood frequency distributions, informed by MCMC output from RMC-BestFit).

1.3 Relationship to RMC-RFA

`rfaR` reimplements the core RMC-RFA methodology in open-source R, offering:

- Reproducibility and transparency in the routing and sampling logic
- Integration with the broader R/RMC ecosystem (e.g., RMC-BestFit outputs)

¹In risk analysis, the *hazard* is the risk source where potential consequences relate to harm (Aven et al., n.d.). In the context of dam and levee risk analyses, the source of risk is often defined using a reservoir stage-frequency relationship. The terms *hydrologic loading*, *stage-frequency*, and *hydrologic hazard* are used interchangeably to relate reservoir/river stages to annual exceedance probabilities (AEPs). These relationships are commonly presented as curves and are used as probabilistic loading conditions in dam and levee risk analyses.

1.4 How This Manual Is Organized

- **Methodology** — statistical and hydrologic methods behind the two-loop simulation framework
- **Installation** — getting `rfaR` running locally
- **Workflow** — an end-to-end walkthrough from raw inputs to a stage-frequency curve
- **Simulation Modes** — Median, Expected, and Full Uncertainty simulation modes explained
- **Function Reference** — Function references (auto-generated from package documentation)
- **Validation** — comparison against RMC-RFA / HEC-HMS benchmark results

For function-level documentation, also see the [pkgdown reference site](#).

2 Methodology

2.1 Overview

`rfaR` implements a two-loop, nested Monte Carlo framework to propagate both natural variability and knowledge uncertainty through a deterministic flood routing model.

2.2 Outer Loop: Parameter Uncertainty

2.3 Inner Loop: Realizations

2.4 Flood Routing

3 Installation

3.1 Requirements

- R (version *TODO*)
- Recommended: [Positron](#) or RStudio

3.2 Installing rfaR

```
# From GitHub  
remotes::install_github("USACE-RMC/rfaR")
```

TODO: note CRAN install instructions once published.

3.3 Dependencies

TODO: list key dependencies (ggplot2, future/future.apply, lmom, cli, etc.) and any that require special handling on USACE machines.

3.4 Verifying Installation

```
library(rfaR)  
# TODO: smoke-test
```

4 Workflow

4.1 Overview

4.2 Step 1: Prepare Inputs

4.3 Step 2: Hydrograph Setup

4.4 Step 3: Sampling

4.5 Step 4: Routing

4.6 Step 5: Running the Full Simulation

4.7 Step 6: Interpreting Results

5 Function Reference

5.1 Execute Reservoir Frequency Analysis

5.1.1 `rfa_simulate()`

RFA Stage-Frequency Simulation

Description:

Performs reservoir stage frequency analysis using Monte Carlo simulation to develop stage-frequency relationships. Combines stratified flow-frequency sampling, flood seasonality, hydrograph scaling, and Modified Puls reservoir routing to estimate annual exceedance probabilities of reservoir stage.

Usage:

```
rfa_simulate(  
  sim_type = "expected",  
  bestfit_params,  
  dist = "LP3",  
  stage_ts,  
  seasonality,  
  hydrographs,  
  resmodel,  
  Nbins = 50,  
  events_per_bin = 200,  
  routing_dt = 1,  
  Ncores = NULL,  
  sim_name = NULL,  
  results_dir = NULL  
)
```

Arguments:

`sim_type`: Character string specifying the simulation mode. One of `"median"`, `"expected"`, or `"full"`. Default is `"expected"`.

`bestfit_params`: Data frame or matrix of distribution parameters from RMC-BestFit MCMC output. Columns 1-3 are distribution parameters:

- * LP3: mean (log), standard deviation (log), skew (log)
- * GEV: location, scale, shape

Column 4 is log-likelihood (used only in `"median"` mode to identify the posterior mode). For `"expected"` mode, must have at least `Nbins * events_per_bin` rows. For `"full"` mode, each row is treated as an independent realization.

`dist`: Character string specifying the frequency distribution. Either `"LP3"` (Log-Pearson Type III, default) or `"GEV"` (Generalized Extreme Value).

`stage_ts`: Data frame of historical reservoir stage with columns `'date'` (character in M/D/YYYY format) and `'stage'` (numeric, feet).

`seasonality`: Numeric vector of length 12 giving monthly flood occurrence probabilities (relative frequencies). Used to sample the month of each simulated event, which determines the antecedent pool elevation.

`hydrographs`: List of hydrograph data frames as returned by `'hydrograph_setup'`. Each element has columns `'datetime'`, `'hour'`, `'inflow'`, and `'hydrograph_num'`, with attributes `"obs_vol"` (observed max n-day volume) and `"dt"` (timestep in hours). The list must have a `"probs"` attribute containing normalized sampling probabilities.

`resmodel`: Data frame with three columns: elevation (ft), storage (acre-ft), and discharge (cfs). Defines the reservoir model for Modified Puls routing.

`Nbins`: Integer. Number of stratified sampling bins in EV1 space. Default is 50.

events_per_bin: Integer. Number of events sampled per bin. Default is 200. Total simulations per realization = 'Nbins * events_per_bin'.

routing_dt: Numeric. Routing timestep in hours. Passed to 'scale_hydrograph' to resample hydrographs before routing. Supported values are '0.25' (15-min), '1' (1-hour, default), '6' (6-hour), and '24' (24-hour).

Ncores: Integer or 'NULL'. Number of parallel workers for "full" mode. If 'NULL' (default), automatically selects based on available cores (capped at 16). Ignored for "median" and "expected" modes.

sim_name: Optional character string to label the simulation. Used in the output CSV filename. If 'NULL' (default), defaults to "sim". Spaces are replaced with underscores in the filename.

results_dir: Optional path to the directory where results are saved. If 'NULL' (default), creates an 'rfaR_results' folder in the current working directory.

Details:

Three simulation modes are available:

"median" Single realization using the most likely (posterior mode) parameter set from RMC-BestFit. Produces one stage-frequency curve with no uncertainty bounds. Useful for quick screening or debugging.

"expected" Single realization using expected value sampling, where each stratified flow sample is paired 1-to-1 with a different parameter set from the posterior distribution. Produces one stage-frequency curve representing the expected (mean) result integrated across parameter uncertainty.

"full" Nested Monte Carlo with full uncertainty quantification. The outer loop iterates over all parameter sets (one per row of 'bestfit_params'), and each inner loop independently samples natural variability via stratified sampling, seasonality, starting pool, and hydrograph shape. Each

realization produces an independent stage-frequency curve; curves are combined on a common stage grid to yield expected, median, and 5th/95th percentile confidence bounds. The outer loop is parallelized using the 'future' framework.

For all modes, natural variability is captured through stratified sampling in with 'Nbins' bins and 'events_per_bin' events per bin, providing reliable coverage from common events down to approximately $1e-8$ AEP.

The stratified sampling approach divides the AEP range (default 0.99 to $1e-8$) into equal-width bins in EV1 (Gumbel reduced variate) space and samples events uniformly within each bin. This ensures adequate representation of rare flood events that would require orders of magnitude more samples under crude Monte Carlo sampling.

Post-processing uses the law of total probability via 'stage_frequency_curve': for a grid of stage thresholds, the weighted exceedance fraction is computed within each bin and summed across bins.

In "full" mode, each parallel worker independently samples all random inputs (seasonality, starting pool, hydrograph shape, and stratified z-variates) in addition to using its own parameter set. This ensures full Monte Carlo independence across realizations. Each worker post-processes its results into a stage-frequency curve immediately, returning only the curve rather than the full peak stage matrix, keeping memory usage manageable for large numbers of realizations.

Curves from individual realizations are combined by interpolating each onto a common stage grid (spanning the global min/max across all realizations) in $\log_{10}(\text{AEP})$ space, then computing summary statistics across realizations at each stage threshold.

Results are automatically exported as a CSV file named '{sim_name}_{sim_type}_{MM_DD_YY_HHMM}.csv' in the 'results_dir' directory. For example, a median simulation named "John McGraw Dam" run on June 4, 2025 at 2:30 PM would produce 'John_McGraw_Dam_median_06_04_25_1430.csv'.

Value:

A list whose contents depend on 'sim_type':

For "median" and "expected":

stage_frequency Data frame with columns 'stage' and 'AEP' from
'stage_frequency_curve'.

peakStage Matrix of simulated peak stages ('events_per_bin' rows
by 'Nbins' columns).

peakFlow Matrix of simulated peak discharges (same dimensions).

weights Stratified sampling bin weights.

For "full":

stage_frequency Data frame with columns 'stage', 'expected',
'median', 'lower_05', and 'upper_95' on a common stage grid.

all_curves List of per-realization stage-frequency data frames
(each with 'stage' and 'AEP' columns).

aep_matrix Matrix of interpolated AEP values (500 rows by
'Nrealizations' columns) on the common stage grid.

common_stage Numeric vector of the common stage grid (length 500).

Nrealizations Number of parameter set realizations processed.

Nsims_per_realiz Number of simulations per realization.

See Also:

'flow_frequency_sampler', 'flow_frequency_sampler_expected',
'stratified_sampler', 'scale_hydrograph', 'mod_puls_routing',
'stage_frequency_curve', 'hydrograph_setup'

Examples:

```
## Not run:
```

```
# Not run: Expected and Median modes take ~15s; Full uncert
```

```

# runs ~10,000 parameter realizations and takes 3-4 hours on a
# typical machine.

# --- Setup ---
hydros <- hydrograph_setup(jmd_hydro_apr1999, jmd_hydro_jun1965,
                           jmd_hydro_may1955, jmd_hydro_pmf,
                           critical_duration = 2, routing_days = 10)

# --- Expected only (default) ---
results_exp <- rfa_simulate(
  sim_type      = "expected",
  bestfit_params = jmd_bf_parameter_sets,
  stage_ts      = jmd_wy1980_stage,
  seasonality   = jmd_seasonality$relative_frequency,
  hydrographs   = hydros,
  resmodel      = jmd_resmodel,
  sim_name      = "jmd"
)

# --- Median only ---
results_med <- rfa_simulate(
  sim_type      = "median",
  bestfit_params = jmd_bf_parameter_sets,
  stage_ts      = jmd_wy1980_stage,
  seasonality   = jmd_seasonality$relative_frequency,
  hydrographs   = hydros,
  resmodel      = jmd_resmodel,
  sim_name      = "jmd"
)

# --- Full uncertainty (parallelized) ---
results_full <- rfa_simulate(
  sim_type      = "full",
  bestfit_params = jmd_bf_parameter_sets,
  stage_ts      = jmd_wy1980_stage,
  seasonality   = jmd_seasonality$relative_frequency,
  hydrographs   = hydros,
  resmodel      = jmd_resmodel,
  Ncores        = 4,
  sim_name      = "jmd"
)
## End(Not run)

```


5.2 Primary RFA Modules

5.2.1 stratified_sampler()

Stratified Sampler

Description:

Creates stratified bins in standard normal space for use in stratified sampling. Stratification improves sampling efficiency by ensuring adequate coverage of rare events.

Usage:

```
stratified_sampler(  
  minAEP = 1e-08,  
  maxAEP = 0.99,  
  dist = "EV1",  
  Nbins = NULL,  
  Mevents = NULL,  
  verbose = FALSE  
)
```

Arguments:

minAEP: Minimum annual exceedance probability. Default is '1E-8'.

maxAEP: Maximum annual exceedance probability. Default is '0.99'.

dist: Character. Probability space for stratification.
Case-insensitive. Default is "EV1". Invalid values trigger a warning and default to "EV1".

"EV1" Extreme Value Type I (Gumbel) space. Recommended for flood frequency analysis. Allocates more bins to rare events through the transformation $-\log(-\log(1-AEP))$, improving tail estimation efficiency.

"Normal" Standard normal (z-score) space. Uniform bins in z-space. Use when the underlying phenomenon is normally distributed.

'Uniform' Uniform probability space. Equal probability width per bin. Generally inefficient for rare event estimation.

Nbins: Number of stratified bins. Default is '20'.

Mevents: Number of events per bin. Default is '500'.

verbose: Logical. If 'TRUE', prints a completion message summarizing the stratification. Default is 'FALSE'.

Details:

The function divides the probability space into bins using the specified transformation. EV1 transformation is recommended for heavy-tailed distributions common in flood frequency analysis, as it naturally allocates more sampling effort to rare events critical for dam safety assessments.

Value:

A list containing:

normOrd Vector of standard normal ordinates spanning all bins

Weights Vector of probability weights for each bin

Zlower Vector of lower bounds for each bin (standard normal)

Zupper Vector of upper bounds for each bin (standard normal)

Nbins Number of bins

Mevents Number of events per bin

Examples:

```
# Default stratification
strat <- stratified_sampler()

# Custom bins and events
strat <- stratified_sampler(minAEP = 1E-6,
                           maxAEP = 0.5,
```

```

                                dist = "EV1",
                                Nbins = 10,
                                Mevents = 100)

# With verbose message
strat <- stratified_sampler(verbose = TRUE)

```

5.2.2 flow_frequency_sampler()

Flow Frequency Sampler

Description:

Generates a stratified matrix of flow values from a single set of frequency distribution parameters using stratified Monte Carlo sampling. Used internally by 'rfa_simulate()' for median-only and full uncertainty modes.

Usage:

```

flow_frequency_sampler(
  bestfit_params,
  freq_dist = "LP3",
  strat_dist = "ev1",
  Nbin = NULL,
  Mevent = NULL
)

```

Arguments:

bestfit_params: Numeric vector of length 3 containing distribution parameters. For LP3: 'c(mean_log, sd_log, skew_log)'. For GEV: 'c(location, scale, shape)'.

freq_dist: Character. Distribution type. Either "LP3" (default) or "GEV".

strat_dist: Character. Probability space for stratification bins. Passed to 'stratified_sampler()'. One of "ev1" (default), "normal", or "uniform". See 'stratified_sampler()' for details.

Nbin: Integer. Number of stratified bins. Default is '50'.

Mevent: Integer. Number of events per bin. Default is '200'.

Value:

A list containing:

flow Matrix of sampled flow values [Mevent x Nbin]

nbins Number of stratified bins

mevents Number of events per bin

weights Probability weights for each bin from
'stratified_sampler()'

See Also:

'stratified_sampler()', 'qp3()', 'rfa_simulate()'

Examples:

```
# Single parameter set (posterior mode)
params <- c(4.85, 0.39, -0.15)
result <- flow_frequency_sampler(params, freq_dist = "LP3",
                                Nbin = 20, Mevent = 500)

# Dimensions of result
dim(result$flow) # 500 x 20

# Distribution of Sampled Flows
hist(result$flow)
```

5.2.3 flow_frequency_sampler_expected()

Flow Frequency Sampler (Expected Only)

Description:

Generates a stratified matrix of flow values by pairing each

z-ordinate with a different parameter set from the posterior distribution. This collapses the nested Monte Carlo structure into a single pass, simultaneously sampling natural variability (via stratified z-ordinates) and knowledge uncertainty (via varying parameters). Used internally by 'rfa_simulate()' for expected-only mode.

Usage:

```
flow_frequency_sampler_expected(  
  bestfit_params,  
  freq_dist = "LP3",  
  strat_dist = "ev1",  
  Nbin = NULL,  
  Mevent = NULL  
)
```

Arguments:

bestfit_params: Data frame of distribution parameters from RMC-BestFit. Must have 'Nbin * Mevent' rows (one parameter set per z-ordinate). For LP3: columns are mean (log), sd (log), skew (log). For GEV: columns are location, scale, shape.

freq_dist: Character. Distribution type. Either "LP3" (default) or "GEV".

strat_dist: Character. Probability space for stratification bins. Passed to 'stratified_sampler()'. One of "ev1" (default), "normal", or "uniform". See 'stratified_sampler()' for details.

Nbin: Integer. Number of stratified bins. Default is '50'.

Mevent: Integer. Number of events per bin. Default is '200'.

Value:

A list containing:

flow Matrix of sampled flow values [Mevent x Nbin]

nbins Number of stratified bins

mevents Number of events per bin

weights Probability weights for each bin from
'stratified_sampler()'

See Also:

'stratified_sampler()', 'qp3()', 'flow_frequency_sampler()',
'rfa_simulate()'

Examples:

```
# Using a pre-loaded parameter set (all 10,000 parameter sets)
result <- flow_frequency_sampler_expected(jmd_bf_parameter_sets,
                                         Nbin = 20, Mevent = 500)

dim(result$flow) # 500 x 20

# Using bootstrapped parameter samples
jmd_samples <- bootstrap_vfc(
  c(jmd_vfc_parameters$mean_log,
    jmd_vfc_parameters$sd_log,
    jmd_vfc_parameters$skew_log),
  dist = "LP3",
  ERL = jmd_vfc_parameters$erl)

jmd_result <- flow_frequency_sampler_expected(
  jmd_samples$params,
  freq_dist = "LP3",
  Nbin      = 20,
  Mevent    = 500)
```

5.2.4 hydrograph_setup()

Hydrograph Setup for RFA Simulation

Description:

Prepares hydrograph data frames copied from RMC-RFA for use in RFA simulation. Converts date/time columns and adds sequential hour and hydrograph ID columns.

Usage:

```
hydrograph_setup(  
  ...,  
  critical_duration = NULL,  
  routing_days = NULL,  
  weights = NULL  
)
```

Arguments:

...: Data frame representing an input hydrograph with columns:
Ord, Date, Time, Flow (cfs). Copied directly from RMC-RFA.
Date & Time should be 'class() = "character"'

critical_duration: Critical duration in days.

routing_days: Desired length of routing simulation in days.

weights: Optional numeric vector of sampling weights for each
hydrograph. Must be the same length as the number of input
hydrographs. Weights are normalized to probabilities
internally. If NULL (default), all hydrographs are weighted
equally.

Value:

A list of formatted hydrograph data frames, each containing:

datetime Date-time (POSIXct)

hour Hours from start of event

inflow Inflow (cfs)

hydrograph_num Hydrograph ID number for sampling

obs_vol Max n-day inflow volume (stored as an attribute of each
dataframe)

dt Hydrograph timestep (delta time, dt) in hours (stored as an
attribute of each dataframe)

The returned list also has a 'probs' attribute containing the normalized sampling probabilities derived from 'weights'.

Examples:

```
# Setup with equal weights (default)
hydros <- hydrograph_setup(jmd_hydro_apr1999,
                           jmd_hydro_jun1965,
                           jmd_hydro_pmf,
                           critical_duration = 2,
                           routing_days = 10)

# Setup with custom weights (PMF 3x more likely to be sampled)
hydros <- hydrograph_setup(jmd_hydro_apr1999,
                           jmd_hydro_jun1965,
                           jmd_hydro_pmf,
                           critical_duration = 2,
                           routing_days = 10,
                           weights = c(1, 1, 3))

# view normalized probabilities
attr(hydros, "probs")
```

5.2.5 scale_hydrograph()

Scale Hydrograph

Description:

Scales inflow hydrograph given a sampled inflow volume from the volume-frequency curve. Scale factor is defined by the sampled inflow volume and the maximum volume of the corresponding duration. If the native hydrograph timestep differs from the target routing timestep, resampling is applied prior to scaling: finer-to-coarser uses block averaging; coarser-to-finer uses linear interpolation.

Usage:

```
scale_hydrograph(
```



```

    hydrograph_shape,
    observed_volume,
    sampled_volume,
    routing_dt = 1
  )

```

Arguments:

hydrograph_shape: Data frame with two columns: time (hours) and inflow (cfs). Must be in that order.

observed_volume: Maximum n-day inflow volume from input hydrograph shape.

sampled_volume: Sampled inflow volume from volume-frequency curve.

routing_dt: Target routing timestep in hours. Supported values are '0.25' (15-min), '1' (1-hour, default), '6' (6-hour), and '24' (24-hour).

Value:

A data frame with two columns: 'time_hrs' and 'inflow_cfs', at the target routing timestep, with time starting at hour 0.

Examples:

```

# Example hydrograph. Requires pre-processing
hydro_example <- hydrograph_setup(jmd_hydro_jun1965_15min, critical_duration = 2, routing_dt = 1)
hydrograph_shape <- hydro_example[[1]][, 2:3]

# Default 1-hour routing timestep
scaled <- scale_hydrograph(hydrograph_shape,
                           observed_volume = 50000,
                           sampled_volume = 55000)

# 15-min input hydrograph, 1-hour routing timestep
scaled <- scale_hydrograph(hydrograph_shape,
                           observed_volume = 50000,
                           sampled_volume = 55000,
                           routing_dt = 1)

```

5.2.6 mod_puls_routing()

Modified Puls Reservoir Routing

Description:

Performs Modified Puls (level pool) routing of inflow hydrograph given a defined reservoir geometry (Stage (ft), Storage (ac-ft), Discharge (cfs)).

Usage:

```
mod_puls_routing(resmodel_df, inflow_df, initial_elev, full_results = FALSE)
```

Arguments:

`resmodel_df`: Data frame with three columns: elevation/stage (ft), storage (acre-feet), and discharge (cfs). Must be in that order.

`inflow_df`: Data frame with two columns: time (hours) and inflow (cfs). Must be in that order.

`initial_elev`: Starting water surface elevation in feet.

`full_results`: Logical. If 'FALSE' (default), returns only peak stage and discharge. If 'TRUE', returns the complete routing result.

Value:

If 'full_results = FALSE', a named numeric vector with 'peak_stage_ft' and 'peak_discharge_cfs'. If 'full_results = TRUE', a data frame with columns: 'time_hr', 'inflow_cfs', 'elevation_ft', 'storage_acft', and 'outflow_cfs'.

References:

Chow, V.T. (1959). Open-Channel Hydraulics. McGraw-Hill.

Examples:

```

# Example hydrograph. Requires pre-processing
hydro_example <- hydrograph_setup(jmd_hydro_jun1965_15min,
                                  critical_duration = 2,
                                  routing_days = 10)
hydrograph_shape <- hydro_example[[1]][, 2:3]
scaled_hydrograph <- scale_hydrograph(hydrograph_shape,
                                       observed_volume = 50000,
                                       sampled_volume = 55000)

# Peak values only
mod_puls_routing(jmd_resmodel, scaled_hydrograph, initial_elev = 3830)

# Full routing table
jmd_full_routing <- mod_puls_routing(jmd_resmodel,
                                    scaled_hydrograph,
                                    initial_elev = 3830,
                                    full_results = TRUE)

head(jmd_full_routing)

```

5.2.7 stage_frequency_curve()

Compute Stage-Frequency Curve

Description:

Converts a matrix of routed peak stages into a stage-frequency curve using the law of total probability across stratified bins. For each candidate stage, the conditional exceedance probability within each bin is weighted by the bin probability and summed to produce the unconditional annual exceedance probability (AEP).

Usage:

```
stage_frequency_curve(peakStage, weights, stage_bins = 1000)
```

Arguments:

peakStage: Matrix of peak stages [Mevents x Nbins] from routing.

weights: Numeric vector of bin weights from 'stratified_sampler()'.
Must have length equal to 'ncol(peakStage)'.

stage_bins: Numeric value of exceedance stages used to compute stage-frequency in each bin. Default is 1000.

Value:

A data frame with columns:

stage Evaluated stage values

AEP Annual exceedance probability at each stage

See Also:

`'stratified_sampler()'`, `'flow_frequency_sampler()'`,
`'rfa_simulate()'`

5.3 Rejection Sampling

5.3.1 pmf_stage_lognormal()

Parameterize a Three-Parameter Lognormal PMF for Stage

Description:

Computes the parameters of a three-parameter lognormal distribution for use as a probabilistic maximum stage (PMF) in rejection sampling. The shift parameter defines the lower or upper bound of the distribution (typically, the lower value). The function supports two modes: (1) supplying a best estimate and assuming sigma, or (2) supplying a best estimate, low, and high to solve for sigma numerically.

Usage:

```
pmf_stage_lognormal(  
  pmf_shift,  
  pmf_mean,  
  pmf_sigma = NULL,  
  pmf_low = NULL,  
  pmf_high = NULL
```

)

Arguments:

`pmf_shift`: Numeric. Assumed lower-bound of the lognormal distribution. This will define the shift value. Must be less than all of 'pmf_mean', 'pmf_low', and 'pmf_high'.

`pmf_mean`: Numeric. Assumed mean of the shifted PMF stage distribution.

`pmf_sigma`: Numeric. Assumed standard deviation on the log scale. Required if 'pmf_low' and 'pmf_high' are not supplied. Defaults to 'NULL'. Suggested value = 0.5.

`pmf_low`: Numeric. Low estimate of PMF stage (assumed 5th percentile). Optional. If supplied, 'pmf_high' must also be supplied.

`pmf_high`: Numeric. High estimate of PMF stage (assumed 95th percentile). Optional. If supplied, 'pmf_low' must also be supplied.

Value:

A named list with the following elements:

`pmf_shift` Hard lower bound of the distribution.

`pmf_mean` Assumed mean of shifted PMF stage distribution.

`pmf_sigma` Standard deviation on the log scale.

`pmf_mu` Derived location parameter on the log scale.

`pmf_p05` 5th percentile of the PMF stage distribution.

`pmf_p95` 95th percentile of the PMF stage distribution.

Examples:

```
# Mode 1: assume sigma
pmf <- pmf_stage_lognormal(pmf_shift = 239, pmf_mean = 241.9, pmf_sigma = 0.5)

# Mode 2: solve for sigma from low/high estimates
```

```
pmf <- pmf_stage_lognormal(pmf_shift = 239, pmf_mean = 241.9,
                           pmf_low = 239, pmf_high = 245)
```

5.3.2 rejection_sampling_stage()

Rejection Sampling of Stage Bounded by a Probabilistic Maximum Stage

Description:

Draws 'n_samples' stage values from a stage-frequency curve, rejecting any draws that exceed a probabilistic maximum stage (PMF) sampled from a three-parameter lognormal distribution. Accepted samples are ranked and assigned Weibull plotting positions for use in frequency analysis.

Usage:

```
rejection_sampling_stage(
  pmf_stage_LN,
  stage_freq_df = NULL,
  aep = NULL,
  stage = NULL,
  n_samples = 1e+07
)
```

Arguments:

pmf_stage_LN: A named list returned by 'pmf_stage_lognormal' containing the lognormal PMF parameters.

stage_freq_df: A data frame with AEP in column 1 and stage in column 2. Mutually exclusive with 'aep' and 'stage'.

aep: Numeric vector of annual exceedance probabilities. Must be supplied with 'stage'. Mutually exclusive with 'stage_freq_df'.

stage: Numeric vector of stages corresponding to 'aep'. Must be supplied with 'aep'. Mutually exclusive with 'stage_freq_df'.

n_samples: Integer. Number of samples to draw. Defaults to '1e7'.

Value:

A data frame of thinned accepted samples with columns for z-score and stage, suitable for plotting a probabilistically bounded stage-frequency curve. Output is produced by 'thin_samples()'.

See Also:

'pmf_stage_lognormal'

Examples:

```
## Not run:

pmf_ln <- pmf_stage_lognormal(pmf_shift = 239, pmf_mean = 241.9, pmf_sigma = 0.5)

result <- rejection_sampling_stage(
  pmf_stage_LN = pmf_ln,
  stage_freq_df = my_stage_freq_df,
  n_samples     = 1e7
)
## End(Not run)
```

5.4 Additional Utilities

5.4.1 bootstrap_vfc()

Bootstrap option for VFC (RFA-style w/ ERL)

Description:

Generates matrix of parameters using posterior mode/mean and ERL

Usage:

```
bootstrap_vfc(bestfit_postmode, dist = "LP3", ERL = 150, Nboots = 10000)
```

Arguments:

bestfit_postmode: Vector of distribution parameters from RMC-BestFit.
For LP3: columns are mean (log), sd (log), skew (log). For
GEV: columns are location, scale, shape.

dist: Distribution type. Either "LP3" (default) or "GEV".

ERL: psuedo effective record length for bootstrapping

Nboots: number of bootstraps

Value:

A list containing:

params Matrix of bootstrapped parameters

dist Distribution selected (LP3 or GEV)

postmode Posterior mode parameters provided to bootstrap

ERL pseudo effective record length provided to bootstrap

See Also:

'flow_frequency_sampler()'

Examples:

```
# Sample using JMD VFC parameters
jmd_samples <- bootstrap_vfc(c(jmd_vfc_parameters$mean_log,
                              jmd_vfc_parameters$sd_log,
                              jmd_vfc_parameters$skew_log),
                             dist = "LP3",
                             ERL = jmd_vfc_parameters$erl)
```

```
# GEV Example with parent distribution parameters
gev_example <- c(3.0, 1.0, -0.1)
gev_samples <- bootstrap_vfc(gev_example,
                             dist = "GEV",
                             ERL = 200,
                             Nboots = 5000)
hist(gev_samples$params[,1])
```



```
# LP3 Example with parent distribution parameters
lp3_example <- c(3.5, 0.22, 0.1)
lp3_samples <- bootstrap_vfc(lp3_example,
                             dist = "LP3",
                             ERL = 300,
                             Nboots = 1000)

hist(lp3_samples$params[,3])
```

5.4.2 qp3()

Pearson Type III Inverse CDF (Quantile Function)

Description:

Computes quantiles from a Pearson Type III distribution given probabilities and distribution parameters (mean, standard deviation, skewness).

Usage:

```
qp3(p, mu, sigma, gamma)
```

Arguments:

p: Vector of probabilities (between 0 and 1).

mu: Mean of the distribution.

sigma: Standard deviation of the distribution.

gamma: Skewness coefficient of the distribution.

Details:

NOTE: This is the non-vectorized version of the function (it will be used in a loop) Vectorized version is part of future development.

The Pearson Type III distribution is parameterized by mean ('mu'), standard deviation ('sigma'), and skewness ('gamma'). These are converted internally to location, scale, and shape parameters.

When skewness is near zero (`'abs(gamma) < 1E-3'`), the normal distribution is used as an approximation.

Value:

Vector of quantiles corresponding to the input probabilities.

Examples:

```
# Single quantile
qp3(0.99, mu = 10, sigma = 2, gamma = 0.5)

# Multiple quantiles
qp3(c(0.5, 0.9, 0.99), mu = 10, sigma = 2, gamma = 0.5)

# Use with JMD volume-frequency parameters
qp3(0.99,
    mu = jmd_vfc_parameters$mean_log,
    sigma = jmd_vfc_parameters$sd_log,
    gamma = jmd_vfc_parameters$skew_log)
```

5.4.3 aep2stage()

Interpolate Stage from AEP

Description:

Given a stage-frequency curve, interpolates stage values at target AEPs. Interpolation is performed in standard normal (z-variate) space for better behavior across orders of magnitude in AEP.

Usage:

```
aep2stage(aep, stage, interp_aep)
```

Arguments:

aep: Numeric vector of annual exceedance probabilities from the known curve.

stage: Numeric vector of stage values corresponding to 'aep'.
interp_aep: Numeric vector of target AEPs to interpolate at.

Value:

Numeric vector of interpolated stage values (same length as 'interp_aep').

See Also:

'stage2aep' for the inverse operation.

Examples:

```
# Get stage at the 1.12% and 0.175% AEP
aep2stage(jmd_rfa_expected$AEP, jmd_rfa_expected$Expected, c(0.0112,1.75E-3))
```

5.4.4 stage2aep()

Interpolate AEP from Stage

Description:

Given a stage-frequency curve, interpolates AEP values at target stages. Interpolation is performed in standard normal (z-variate) space for better behavior across orders of magnitude in AEP.

Usage:

```
stage2aep(aep, stage, interp_stage)
```

Arguments:

aep: Numeric vector of annual exceedance probabilities from the known curve.

stage: Numeric vector of stage values corresponding to 'aep'.

interp_stage: Numeric vector of target stage values to interpolate at.

Details:

Stage-frequency curves typically flatten at very rare AEPs once the reservoir reaches the reservoir model discharge capacity. This produces tied stage values. 'approx()' handles these by averaging the corresponding AEPs, which may emit a "collapsing to unique 'x' values" warning. This is expected behavior.

Value:

Numeric vector of interpolated AEP values (same length as 'interp_stage').

See Also:

'aep2stage' for the inverse operation.

Examples:

```
# Get AEP at stage 3862.0-ft and 3872-ft
stage2aep(jmd_rfa_expected$AEP, jmd_rfa_expected$Expected, c(3862.0, 3872))
```

5.4.5 interpolate_aep_matrix()

Interpolate AEPs at Target Stages Across Realizations

Description:

For each realization's stage-frequency curve, interpolates the AEP at every target stage value. Interpolation is performed in standard normal (z-variate) space.

Usage:

```
interpolate_aep_matrix(results_list, target_stages)
```

Arguments:

results_list: List of realization data frames, each with columns 'stage' and 'AEP'.

target_stages: Numeric vector of stage values to interpolate at.

Value:

Matrix of AEP values with rows = target stages and columns = realizations.

5.4.6 interpolate_stage_matrix()

Interpolate Stages at Target AEPs Across Realizations

Description:

For each realization's stage-frequency curve, interpolates the stage at every target AEP value. Interpolation is performed in standard normal (z-variate) space.

Usage:

```
interpolate_stage_matrix(results_list, target_aeps)
```

Arguments:

results_list: List of realization data frames, each with columns 'stage' and 'AEP'.

target_aeps: Numeric vector of AEP values to interpolate at.

Value:

Matrix of stage values with rows = target AEPs and columns = realizations.

5.4.7 power_function()

Power Function

Description:

Raises 10 to the 1st through the nth power

Usage:

```
power_function(x)
```

Arguments:

x: Highest power of 10 to use

Value:

Vector of 10 raised sequentially from the 1st to the nth power.

5.4.8 theme_rfar_conceptual()

rfaR ggplot2 Theme for Conceptual Plots

Description:

A custom 'ggplot2' theme used in the rfaR conceptual realization vignette. Based on 'theme_bw' with tightened text sizes, italic titles, and the legend suppressed. Intended for figures that emphasize curves over annotation density.

Usage:

```
theme_rfar_conceptual()
```

Value:

A 'ggplot2' theme object that can be added to a 'ggplot' via '+'.

See Also:

'theme_bw'

Examples:

```
library(ggplot2)
ggplot(jmd_rfa_expected, aes(x = AEP, y = Expected)) +
  geom_line() +
  scale_x_continuous(transform = c("log10", "reverse")) +
```

```
labs(title = "JMD Expected Stage-Frequency",  
      subtitle = "Demonstrating theme_rfar_conceptual()",  
      x = "AEP", y = "Stage (ft)") +  
theme_rfar_conceptual()
```

6 Simulation Modes

`rfaR` supports three simulation modes, corresponding to different treatments of parameter uncertainty.

6.1 Median Mode

6.2 Expected Mode

6.3 Full Uncertainty Mode

Runs the full nested Monte Carlo framework, propagating both parameter (epistemic) and event (aleatory) uncertainty.

6.4 Selecting a Mode

7 Validation

7.1 Purpose

7.2 Modified Puls Routing

7.3 Stratified Sampling

7.4 See Also

Detailed validation vignettes are also available on the [pkgdown site](#).

Aven, Terje, Yakov Ben-Haim, Henning Boje Andersen, et al. n.d. *Society for Risk Analysis Glossary*.